salesianos TRIANA | Colegio San Pedro

salesianos

TRIANA

Proyecto Intermodular de Desarrollo de Aplicaciones Multiplataforma

Junio 2026

Versión 1.0

HOJA DE CONTROL DE CAMBIOS

Versión	Descripción	Fecha
1.0	Versión inicial del documento	04/03/2025

Portada

Índice

Portada 3
Índice 3
1. Introducción 3
2. Plan de Empresa 4
JUSTIFICACIÓN: 4
NOMBRE y LOGO 4
PRODUCTO 4
MODELO DE NEGOCIO 4
CONSUMIDORES Y POSIBLES CLIENTES 4
COMPETENCIA 5
RENTABILIDAD 5
3. Descripción detallada del sistema y listado de historias de usuario 5
4. Modelado y diseño 5
5. Diseño 6
6. Implementación 8
6. Implementación 8
6.1 Backend API (sacristan-api) 8
6.2 Frontend Web (sacristan-web) 8
6.3 Aplicación Móvil (sacristan-app) 9

1. Introducción

El presente proyecto consiste en el diseño y desarrollo de una aplicación multiplataforma (dispositivos móviles y entorno web) orientada a la creación y gestión de rutinas de acciones a través de pictogramas. Esta herramienta está dirigida principalmente a personas con trastornos del espectro del autismo (TEA), parálisis cerebral, discapacidad intelectual u otras condiciones que requieran el uso de Sistemas Aumentativos y Alternativos de Comunicación (SAAC).
El objetivo principal es ayudar a compensar las dificultades de comunicación y comprensión del entorno, proporcionando a los usuarios una forma visual y sencilla de anticipar sus tareas diarias, lo que reduce el estrés causado por los cambios imprevistos y favorece su autorregulación y autonomía

2. Plan de Empresa

JUSTIFICACIÓN

La necesidad de estructurar el día a día es vital para las personas con TEA. Aunque existen herramientas de comunicación aumentativa, este proyecto destaca por ofrecer una solución integral que conecta a alumnos, profesores y padres. La aplicación aporta la novedad de sincronizar la gestión de secuencias en un entorno educativo (Instituto Doctor Sacristán) , integrando directamente los recursos y pictogramas de ARASAAC , y generando estadísticas detalladas sobre el rendimiento y los tiempos de ejecución de cada alumno.

NOMBRE y LOGO

El nombre refleja el entorno donde nace y se aplica inicialmente el proyecto (Instituto Doctor Sacristán). El logotipo hace uso de colores amigables (azul ilustrado) e iconografía clara para ser accesible a usuarios con necesidades de apoyo visual.

PRODUCTO

Una **aplicación móvil** (Android/iOS) para los alumnos, donde visualizan y reproducen de forma guiada (paso a paso con porcentaje de avance) sus rutinas diarias mediante pictogramas y alarmas de tiempo.
Un **panel web / app para profesores y administradores**, que permite crear tareas, gestionar usuarios y asignar secuencias a los alumnos.
Una **interfaz para padres**, centrada en visualizar el calendario de su hijo y el contacto con el profesor.
Finalidad: Cubrir la necesidad de anticipación y organización temporal del alumnado con TEA. (Aquí deberás insertar las capturas de pantalla de Figma que tienes en la presentación, como la pantalla del alumno y la de reproducción de secuencias)

MODELO DE NEGOCIO

El modelo se enfoca principalmente en el sector educativo (B2B - *Business to Business*), ofreciendo la plataforma como un servicio (SaaS) a colegios, institutos y centros de educación especial para que gestionen las rutinas de sus estudiantes. Opcionalmente, podría ofrecerse directamente a familias o terapeutas independientes.

CONSUMIDORES Y POSIBLES CLIENTES

Usuarios finales (Consumidores): Niños y jóvenes con TEA u otras dificultades del lenguaje y la comunicación.

- *Clientes (Compradores):* Instituciones educativas públicas, concertadas y privadas (como el Instituto Dr. Sacristán), terapeutas especializados y asociaciones de apoyo al autismo.

COMPETENCIA

Existen herramientas en el mercado de la comunicación aumentativa (SAAC), como *LetMeTalk* o *DictaPicto*. Sin embargo, nuestra aplicación se diferencia al estar altamente enfocada en la **gestión de rutinas guiadas por tiempo** (con alarmas y franjas horarias) y en la analítica de progreso del usuario (tiempo empleado por paso, pulsaciones, etc.) para el profesorado.

RENTABILIDAD

La aplicación generará ingresos mediante la venta de licencias de uso a centros educativos (pago anual o mensual por número de alumnos matriculados en la plataforma) o mediante planes de suscripción "Premium" para padres que deseen funcionalidades de seguimiento avanzadas en casa

3. Descripción detallada del sistema y listado de historias de usuario

Proyecto de github con las tareas y la organización

4. Modelado y diseño

4.1 Prototipado en Figma de la aplicación

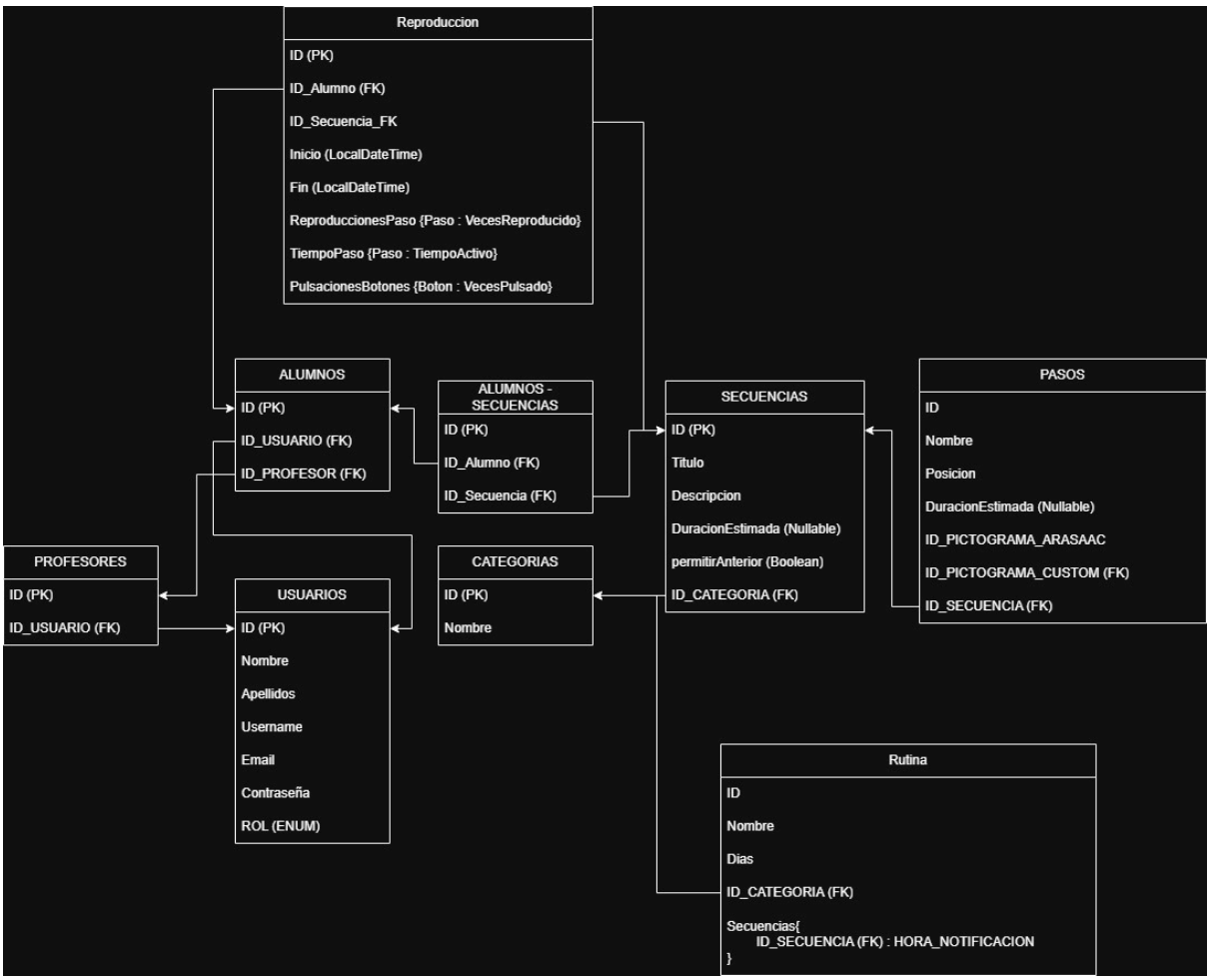
Prototipo inicial Figma

Prototipo inicial Figma 2

Ayuda de diseño web y app legacy Figma Make

Ayuda de diseño web 2.0 Figma Make

4.2 Diagrama de clases del modelo de dominio de la API.



5. Diseño

5.1 Diagrama de clases del diseño indicando: direccionalidad de las asociaciones, tipos y los tipos de datos a utilizar.

User (Usuario): Entidad base para el control de acceso y autenticación.

- *Atributos:* id (Long), name (String), lastName (String), username (String, único), email (String, único), password (String), boolean de control de cuenta y timestamps (createdAt, lastPasswordChangeAt). Contiene un conjunto de roles (Set<Role>).
- *Relaciones:* Mantiene una relación **1:1** bidireccional (a través de la anotación @MapsId) con las entidades Teacher y Student, compartiendo el mismo identificador (id).

Teacher (Profesor): Entidad que representa al personal docente o administrador.

- *Relaciones:* Tiene una relación **1:N** con Student (un profesor puede tener asignados múltiples alumnos, pero un alumno tiene un solo profesor principal responsable).

Student (Alumno): Entidad que representa al usuario final que reproducirá las secuencias.

- *Relaciones:* * **N:1** hacia Teacher.
    - **N:M** con Routine (un alumno puede tener varias rutinas, y una rutina puede ser asignada a varios alumnos).
    - **N:M** con Sequence (relación para las secuencias sueltas o de biblioteca asignadas al alumno).

Category (Categoría): Agrupa secuencias y rutinas por temáticas (ej. Higiene, Comedor).

- *Relaciones:* **1:N** hacia Sequence y **1:N** hacia Routine.

Sequence (Secuencia): Representa una acción principal (ej. "Lavarse las manos").

- *Atributos:* title (String), description (String), estimatedDuration (Duration), allowGoBack (Boolean).
- *Relaciones:* **1:N** bidireccional hacia Step (una secuencia se compone de múltiples pasos) y **N:1** hacia Category.

Step (Paso): Acción atómica dentro de una secuencia.

- *Atributos:* name (String), position (Integer, para el orden), estimatedDuration (Duration), arasaacPictogramId (Integer, para enlazar con la API externa de pictogramas).
- *Relaciones:* **N:1** hacia Sequence.

Routine (Rutina): Conjunto estructurado de secuencias que se repiten en ciertos días.

- *Atributos:* name (String), daysOfTheWeek (Set de Enum para los días de repetición).
- *Relaciones:* **1:N** hacia RoutineSequence.

RoutineSequence (Secuencia de Rutina - Clase Intermedia): Rompe la relación N:M entre Routine y Sequence añadiendo información de contexto temporal.

- *Atributos:* startTime (LocalTime), endTime (LocalTime). Define en qué franja horaria debe ejecutarse una secuencia dentro de una rutina.

Reproduction (Reproducción - Estadísticas): Entidad analítica que registra cómo un alumno ejecuta una secuencia.

- *Atributos:* startedAt, endedAt, y varios Mapas (Map) para registrar cuántas veces se reprodujo cada paso, el tiempo empleado en milisegundos por paso, y el conteo de clics en la interfaz.
- *Relaciones:* **N:1** hacia Student y **N:1** hacia Sequence.

6. Implementación

El proyecto se ha desarrollado bajo una arquitectura de microservicios lógicos separada en tres repositorios principales, garantizando el principio de responsabilidad única.

6.1 Backend API (sacristan-api)

Desarrollada en **Java 21** con el framework **Spring Boot**. Sigue un patrón de arquitectura por capas (Controlador, Servicio, Repositorio) pero organizada en paquetes según su dominio y el cliente que los consume:

- com.sacristan.api.global: Contiene la configuración transversal del sistema. Aquí se alojan los Modelos de datos (models), los Repositorios JPA (repositories), la configuración de Seguridad con JWT (security/config/jwt), el manejo global de excepciones (error) y las validaciones personalizadas (validation).
- com.sacristan.api.interfaces.admin: Expone los Endpoints (Controladores) y la Lógica de Negocio (Servicios) orientados exclusivamente al panel web del profesorado. Sigue el patrón DTO (*Data Transfer Object*) para no exponer las entidades reales (dtos/category, dtos/student, etc.) y dispone de operaciones CRUD completas.
- com.sacristan.api.interfaces.student: Módulo altamente optimizado para la aplicación móvil. Los controladores y servicios de este paquete están diseñados para devolver únicamente la información estrictamente necesaria para que el alumno reproduzca las secuencias (ej. PrincipalSequenceDTO, PlaySequenceDTO), reduciendo el consumo de datos y mejorando el rendimiento.

6.2 Frontend Web (sacristan-web)

Panel de administración desarrollado en **Angular** y **TypeScript**. Su estructura se divide en:

- src/app/logic: Contiene el núcleo lógico del cliente web. Incluye los guards e interceptors para la protección de rutas y la inyección del token JWT en las peticiones. También se definen las interfaces que mapean los DTOs del backend y los services encargados de la comunicación HTTP (incluyendo el arasaac-service.ts para conectar con la API de pictogramas).
- src/app/views/components: Arquitectura basada en componentes reutilizables. Incluye elementos genéricos como botones (action-buttons), modales para formularios (student-form-modal), listas de elementos (sequence-row, teacher-row) y barras de carga (loading-component).
- src/app/views/pages: Vistas principales enrutables de la aplicación, como la página de login (login-page) y la página de gestión de contenidos (content-list-page).

6.3 Aplicación Móvil (sacristan-app)

Aplicación multiplataforma (Android/iOS) desarrollada en **Flutter** utilizando el lenguaje **Dart**. Utiliza el patrón **BLoC** (Business Logic Component) para la gestión del estado, separando la interfaz de usuario de la lógica de negocio. Su estructura se organiza por funcionalidades (*Feature-First*):

- lib/core: Contiene los modelos de datos, la gestión de la sesión (token_storage.dart), los servicios de conexión a la API (auth_service, sequence_service) y el manejo de excepciones de red.
- lib/features: Cada carpeta dentro de este directorio es un módulo independiente con su propio estado (Bloc, Event, State) y su Interfaz Gráfica (UI / Widgets). Destacan funcionalidades críticas como play_sequence (encargada de la lógica de la barra de progreso, la reproducción por pasos y el registro de estadísticas) y sequences_to_do (horario diario del alumno).
- lib/pages y lib/shared: Contienen las vistas contenedor principales (main_view) y componentes visuales compartidos por toda la aplicación, como la barra de navegación inferior (custom_bottom_navigation_bar_item).